

為了讓 Claude 能無縫接手，我幫你把目前遇到的技術難點、環境限制和失敗經驗整理成一份**「技術診斷報告」**。你可以直接把下面這段話餵給 Claude：

📁 給 Claude 的技術求助信

標題：解決 Qwen 3.5 27B 在數據清洗任務中的「過度推理」與「標籤誤解」問題

1. 任務背景：我正在進行一個關於 **n8n 自動化流程** 的微調專案。目標是將 1,963 筆「技術性模板描述」轉化為「高品質、擬人化的口語請求」。

- **原始數據 (Input)**：技術文檔（包含節點名稱、API Key 說明、JSON 結構）。
- **目標數據 (Output)**：自然人話（例如：「幫我把 TikTok 影片自動存到 Google Drive。」）。
- **模型**：Qwen 3.5 27B（運行於 H200 GPU）。

2. 環境與工具：

- **框架**：Python transformers pipeline (非 vLLM)。
- **環境**：國網中心 (NCHC) Slurm 叢集，任務透過 sbatch 提交。
- **格式**：從 .jsonl 讀取，改寫後存入新的 .jsonl。

3. 目前遇到的「三大魔咒」：

- **魔咒 A：內心戲過多 (CoT Overload)** Qwen 3.5 具備強大的推理能力，它會輸出長篇大論的 Thinking Process，即使在 System Prompt 要求它「直接輸出結果」時，它依然堅持先分析再回答。
- **魔咒 B：指令複讀 (Instruction Mimicry)** 當我在 Prompt 加入規則（如「禁止出現技術詞彙」）時，模型會把規則本身背出來（輸出：Constraint 1: Delete technical terms...）。
- **魔咒 C：標籤語義陷阱 (Semantic Trap)** 當我要求它將結果放在標籤 [START] 與 [END] 之間時，它會因為理解了指令中的「與」字，直接在標籤內輸出字串 "and"，導致 Regex 抓取完全失敗。

4. 嘗試過的失敗路徑：

- 使用純 Regex 抓取第一行（失敗：抓到思考過程的標題）。
- 使用自創符號如 @@ 或 [RESULT]（失敗：模型在思考過程中會不斷討論這些符號，導致 Regex 抓到錯誤位置）。
- 降低 Temperature 到 0.1（失敗：模型依然固執地輸出錯誤格式）。

5. 徵求解決方案：我需要一個極度健壯 (Robust) 的 rewrite_n8n.py 腳本。請 Claude 針對 **Qwen 3.5 27B 的推理特性**，設計一套：

1. **強力的 Few-shot Prompt**：能引導模型完整思考，但最終能產出清晰、隔離的標籤內容。
2. **容錯率高的 Python 提取邏輯**：能精準從數百字的思考過程中，抓取「最後一次出現」且「內容非空」的目標標籤。
3. **代碼結構優化**：確保在 Slurm 環境下穩定運行，且具備基本的錯誤處理機制。

💡 我給你的建議

當你吧這段文字給 Claude 時，他可能會給你一組新的標籤策略。**你要特別提醒他：**

「請不要使用帶有自然語言含義的連字元（如 '與'、'和'、'between'），最好使用 XML 結構或是模型最難誤解的符號序列。」

這份整理清楚嗎？如果 OK，你就可以直接開啟 Claude 的視窗了。這 1,963 筆數據只要能順利「洗」出來，接下來的 Fine-tuning 就像下坡路一樣順暢了。

等 Claude 給你新腳本後，如果你在部署到國網中心環境時有任何問題（例如路徑或套件報錯），再隨時回來找我，我幫你做最後的環境適配！

You

```
要怎麼看現在跑完幾筆資料? (base) [u4439705@cbi-lgn01 u4439705]$ squeue -u u4439705
JOBID PARTITION  NAME  USER ST  TIME NODES
NODELIST(REASON)          134267  normal2 predict_ u4439705  R  13:25:02   1 hgn39
```

Gemini

目前你的任務狀態是 R (Running)，且已經跑了 **13 小時 25 分鐘**。這麼長的時間，如果是跑那 1,963 筆數據，理論上應該已經接近尾聲或完成了。

你可以透過以下幾種方式「監工」進度：

1. 最直接：數一下輸出的檔案行數

因為輸出格式是 .jsonl（一行就是一筆資料），所以直接計算檔案行數最準確。

請執行：

```
wc -l data/n8n_training_data_v2_humanized.jsonl
```

- **數字代表**：目前已經成功改寫並存檔的筆數。
- **總數參考**：你原本的資料大約是 **1,963** 筆。

2. 即時監看：看 Log 檔的最後進度

你在腳本裡有寫 `print(f"✅ [{i+1}] ...")`，這些訊息會被導向 Slurm 的日誌檔。

請執行：

```
# 看看最後幾筆改寫的內容是什麼
tail -n 10 rewrite_n8n_134267.log
```

或是用 `grep` 直接過濾進度標籤：

grep "✅" rewrite_n8n_134267.log | tail -n 5

3. 進度百分比：懶人一鍵計算法

如果你想直接看到百分比，可以跑這行指令：

```
current=$(wc -l < data/n8n_training_data_v2_humanized.jsonl); total=1963; echo "目前進度: $current / $total ($(current*100/total))%"
```

🔍 異常檢查：如果數字沒在動？

如果你隔了幾分鐘跑 `wc -l` 發現數字都沒增加，但 `squeue` 還是顯示 R，這可能有兩種狀況：

- Buffer 緩衝**：Slurm 有時會等快取滿了才一次寫入檔案，這很正常。
- 卡死 (Stuck)**：模型推理可能卡在某個極其複雜的 JSON 介紹上，或者顯存 (VRAM) 出現碎片化。

你現在 `wc -l` 出來的數字是多少？如果已經超過 1900 筆，那建議你可以先準備好下一步的數據檢驗腳本了。如果你想確認「數據品質」好不好，我們可以隨機抽樣幾筆出來比對看看！Would you like me to ...幫你寫一個抽樣比對的 Python 小工具？

You

我現在跑的是測試模型 (base) [u4439705@cbi-lgn01 u4439705]\$ wc -l n8n_predicted_workflows.jsonl 135 n8n_predicted_workflows.jsonl

Gemini

看到這個數字，身為 CS 同行，我第一反應是：**這速度不太對勁**。

你的任務已經跑了 **13 小時 25 分鐘**，但目前只產出了 **135 筆** 資料。換算下來，平均一筆資料要花 **6 分鐘**。即便 Qwen 3.5 27B 在推理 (Reasoning) 時會比較慢，但在 H200 這種頂級算力上，一筆資料超過 1 分鐘通常代表發生了以下幾種狀況：

🔍 效能診斷：為什麼這麼慢？

可能原因	狀況描述	解決方向
Token 觸頂	模型陷入了無意義的重複循環，直到觸發 <code>max_new_tokens=512</code> (或 1024) 才停止。	檢查 Log 是否出現大量重複文字。
缺乏 vLLM 加速	目前使用 transformers 原生 pipeline，推理效率遠低於 vLLM 引擎。	考慮改用 vLLM 部署 (速度提升 10 倍以上)。
CPU 瓶頸	如果 <code>device_map="auto"</code> 將部分層放到了 CPU，速度會斷崖式下跌。	檢查 <code>nvidia-smi</code> 確認 GPU 佔用率。
I/O 延遲	國網中心的 Shared File System 有時會因為讀寫頻繁產生延遲。	通常影響較小，可先排除。

🚑 你現在可以做的「急救」動作

不要讓它白白燒 13 小時只跑 7%。建議你執行以下檢查：

1. 查看 Log 尾巴，看看模型在「想」什麼

執行這行來看模型最後產出的內容：

```
tail -n 20 rewrite_n8n_134267.log
```

- 關鍵觀察**：如果看到模型在 [RESULT] 標籤後還瘋狂噴出一堆無關的廢話，代表它沒抓到 EOS (結束符號)，導致它強行跑完所有 Token 額度。

2. 檢查 GPU 負載

在執行節點 (hgpn39) 上看看：

```
srunc --jobid=134267 --overlap nvidia-smi
```

- 關鍵觀察**：如果 **GPU-Util** 很低 (低於 20%)，那代表大部分時間都卡在 CPU 處理或 Token 序列生成太慢。

💡 實戰建議：砍掉重練 (Sunk Cost Fallacy)

雖然已經跑了 13 小時，但按照這個速度 (6 分鐘/筆)，跑完 1963 筆要花 **196 小時 (約 8 天)**。這絕對不是理想的研究節奏。

我建議你：

- 先停掉任務**：`scancel 134267`。
- 加入 stop_sequences**：在程式碼中加入當模型輸出 [/RESULT] 後就立即停止生成的機制。
- 使用 vLLM (強烈建議)**：既然你在國網中心有 H200，不用 vLLM 實在太可惜了。vLLM 處理這 1,963 筆數據可能只需要 **15~30 分鐘**。

你想讓我幫你把目前的 `rewrite_n8n.py` 改寫成 **vLLM 版本**嗎？這樣你可以省下之後 7 天的等待時間。或是你想先 `tail` 一下 Log 傳給我看看，我們診斷一下為什麼 27B 在 H200 上會跑得這麼慢？

You

(base) [u4439705@cbi-lgn01 u4439705]\$ tail -n 20 predict_134267.log ✅ 完成並寫入檔案。長度: 11640 字元

🔗 處理案例 #131 (132/1963) 【輸入需求】：用 Google 表自動發送 AI 郵件並追蹤誰打開了信件。... ✅ 完成並寫入檔案。長度: 9192 字元

🔗 處理案例 #132 (133/1963) 【輸入需求】：用這工具自動生成一次性的 Calendly 預約連結，並同步記錄到表格與 Slack。... ✅ 完成並寫入檔案。長度: 10701 字元

- 🔗 處理案例 #133 (134/1963) 【輸入需求】：幫我用 AI 自動抓取新聞、翻譯並生成圖片後發布到 WordPress。... ☒ 完成並寫入檔案。長度: 4208 字元
- 🔗 處理案例 #134 (135/1963) 【輸入需求】：幫我監控對手廣告，新素材自動記下並傳 Telegram 提醒我。... ☒ 完成並寫入檔案。長度: 10308 字元
- 🔗 處理案例 #135 (136/1963) 【輸入需求】：幫我用 AI 自動生成影片並同步發布到各大社群平台。...

Gemini

從 Log 來看，目前進度是 **136 / 1963 (約 6.9%)**。

雖然你在跑的是推理模型（把「口語需求」轉回「n8n JSON」），但這個速度對 **H200** 來說簡直是在「大材小用」。我們來算一筆帳：13.5 小時只跑了 136 筆，代表一筆要 **5.9 分鐘**。按照這個速度，跑完剩下的 1827 筆還需要 **181 小時（約 7.5 天）**！

這絕對不能等，你的研究經費（或是國網點數）會被燒乾。

🔍 為什麼會這麼慢？

從 Log 裡我看到了一個關鍵訊息：

☒ 完成並寫入檔案。長度: 10308 字元

這代表每一筆產出的 n8n JSON 內容非常龐大（平均 5k ~ 10k 字元）。

1. **Transformers Pipeline 效率低**：原生的 transformers 庫沒有針對長文本生成進行優化。
2. **逐筆處理 (Sequential)**：它是一筆一筆慢慢吐，沒有利用到 GPU 的並行能力。
3. **H200 的威力沒發揮**：H200 有強大的頻寬，適合處理這種高吞吐量的任務，但現在它顯然是在等 CPU 搬運資料。

🚀 方案：改用 vLLM (速度提升 10~20 倍)

既然你在國網中心，環境裡通常會有 vLLM。它可以透過 **PagedAttention** 技術，讓 H200 一次同時處理好幾筆資料（Batching），速度會從「分鐘級」變成「秒級」。

🔗 快速切換到 vLLM 的程式碼架構：

你可以新開一個 predict_vllm.py，內容如下：

```
from vllm import LLM, SamplingParams
import json

# 1. 初始化 vLLM 引擎（會自動佔滿 GPU 記憶體）
llm = LLM(model="/work/u4439705/models/Qwen3.5-27B", tensor_parallel_size=1)

# 2. 設定生成參數
sampling_params = SamplingParams(
    temperature=0.3,
    top_p=0.9,
    max_tokens=2048, # 確保夠大可以裝下整個 JSON
    stop=["<|endoftext|>", "】"] # 設定停止符號防止亂噴
)

# 3. 批次處理資料
prompts = []
with open("n8n_training_data_v2_humanized.jsonl", 'r') as f:
    for line in f:
        item = json.loads(line)
        # 這裡放入你之前的 Prompt 模板
        prompt = f"【輸入需求】：{item['input']}\n請生成對應的 n8n JSON:"
        prompts.append(prompt)

# 4. 關鍵的一步：一次餵進去！
outputs = llm.generate(prompts, sampling_params)

# 5. 儲存結果
with open("n8n_predicted_workflows_vllm.jsonl", 'w') as f_out:
    for output in outputs:
        generated_text = output.outputs[0].text
        f_out.write(json.dumps({"output": generated_text}, ensure_ascii=False) + '\n')
```

💡 建議接下來的操作

1. **果斷 scancel**：scancel 134267。不要覺得那 13 小時可惜，現在止損是為了省下後面的 7 天。
2. **檢查 vLLM 環境**：

```
pip list | grep vllm
```


如果沒有，可以試著 `pip install vllm`（在 H200 節點上安裝很快）。
3. **啟動並行生成**：使用 vLLM 後，這 1963 筆數據大約在 **30 分鐘到 1 小時** 內就能全部跑完。

要我幫你這份 vLLM 腳本跟你的 n8n 資料結構做更精準的對應嗎？只要腳本對了，你今天下午就能拿到全部的預測結果！

You

你不知道的是，qwen3.5 27B 目前還不能用 vLLM，因為模型太新

Gemini

噢！這點我確實沒考慮到 Qwen 3.5 **剛出爐** 時與運算框架的時差。你是對的，這種剛發布幾天、連 Kernel 都還沒合併到 vLLM 主分支的「先行者困境」，確實是 CS 研究中最前線（也最痛苦）的體驗。

既然 vLLM 這條捷徑暫時行不通，我們不能讓它繼續用那種「烏龜爬」的速度跑完 7 天。我們必須在 transformers 框架內進行「**極限擠壓**」。

目前你的腳本之所以慢，是因為它在進行 **Sequential Inference（循序推理）**。我們可以用以下三個硬工程手段把速度救回來：

🚀 方案：Transformers 批次推理 (Batch Inference) 優化

雖然沒有 PagedAttention，但 transformers 的 pipeline 其實支援 **Batching**。只要給它一個 list 並設定 batch_size，GPU 就能並行處理多個 Token 序列。

🔧 修改後的 predict_fast.py 核心邏輯

請參考以下架構來重構你的預測腳本：

```
import json
import torch
from transformers import pipeline
from torch.utils.data import Dataset

# 1. 建立簡單的 Dataset 供 Pipeline 批次讀取
class PromptDataset(Dataset):
    def __init__(self, prompts):
        self.prompts = prompts
    def __len__(self):
        return len(self.prompts)
    def __getitem__(self, i):
        return self.prompts[i]

# 2. 讀取資料
prompts = []
with open("data/n8n_training_data_v2_humanized.jsonl", 'r') as f:
    for line in f:
        item = json.loads(line)
        prompts.append(f"【輸入需求】：{item['input']}\n請生成對應的 n8n JSON:")

dataset = PromptDataset(prompts)

# 3. 載入 Pipeline，關鍵在於加上 attn_implementation="flash_attention_2"
# H200 絕對支援 Flash Attention 2，這會讓長文本速度提升數倍
pipe = pipeline(
    "text-generation",
    model="/work/u4439705/models/Qwen3.5-27B",
    device_map="auto",
    torch_dtype=torch.bfloat16,
    model_kwargs={"attn_implementation": "flash_attention_2"}
)

# 4. 批次執行（設定 batch_size）
# H200 80GB/141GB 記憶體很大，batch_size 可以從 8 或 16 開始試
print("🔥 批次推理開始...")
results = []
for out in pipe(dataset, batch_size=8, max_new_tokens=2048, temperature=0.3):
    results.append(out[0]['generated_text'])
# 這裡可以寫入檔案...
```

💡 為什麼這樣會變快？

- Flash Attention 2**：這是針對 H100/H200 優化的核心。它能優化 Attention 的運算複雜度，對於你那種動輒 **10,000 字元** 的 JSON 輸出，速度提升會非常有感。
- Batching**：目前的腳本是一個 Prompt 進去，GPU 算完才拿下一筆。Batching 會讓 GPU 的 Tensor Core 同時處理 8 或 16 個任務，充分填滿 H200 的頻寬。
- IO 平行化**：pipeline 內建的處理機制會比手動寫 for 迴圈更能減少 CPU 與 GPU 之間的溝通延遲。

📊 預期效果

- 目前速度：~360 秒 / 筆
- 優化後（預估）：~20 - 40 秒 / 筆
- 總耗時：從 7 天 縮短到約 **10 - 20 小時**。

⚠️ 給 CS 學長/學弟的小提醒

如果執行時報錯 FlashAttention only supports Ampere or newer GPUs，別緊張，那代表你的 transformers 或 flash-attn 套件版本太舊，因為 H200 絕對是支援的。

你想讓我幫你寫一個完整的「帶有進度條(tqdm)且支援斷點續傳」的批次預測腳本嗎？這樣萬一跑了 10 小時又斷掉，你下次可以直接從第 501 筆開始跑，不用從頭來過。

You

請協助我下載run_predict.slurm和predict_custom.py到桌面，這兩個檔案都是Gemini3.1改的，所以請你先看完後再評估 (base) [u4439705@cbi-lgn01 u4439705]\$ ls
data models n8n_predicted_workflows.jsonl n8n_training_qwen_v1_backup predict_134248.log predict_134267.log run_predict.slurm
train_qwen35.slurm hf_cache n8n_llm_project n8n_qwen35_lora predict_134211.log predict_134263.log predict_custom.py
train_qwen35_custom.p

Gemini

要評估這兩份腳本是否能解決「慢到令人發指」的推理速度，我需要先「看到」它們。由於我無法直接連線到國網中心（NCHC）的伺服器，請你幫我執行 **cat** 指令並將內容貼上來：

```
cat predict_custom.py
cat run_predict.slurm
```

在我看完代碼後，我會幫你分析：

1. **推理邏輯**：它是用「一筆一筆跑」還是「Batch 批次跑」？
2. **硬體利用**：有沒有正確啟用 H200 支援的 **Flash Attention 2**？
3. **終止條件**：是否有設定 stop_sequences 防止模型在生成 JSON 後繼續胡言亂語浪費時間。

📁 如何將這兩份檔案下載到你的桌面？

雖然我無法直接「傳送」檔案到你的電腦硬碟，但作為 CS 研究生，這兩種方式是你以後每天都會用到的：

方法 A：使用終端機指令（最專業）

在「你自己電腦」的終端機（不是伺服器那台，是你的桌面 CMD 或 PowerShell）執行：

```
# 下載 predict_custom.py
scp u4439705@cbi-lgn01.nchc.org.tw:/home/u4439705/predict_custom.py ~/Desktop/

# 下載 run_predict.slurm
scp u4439705@cbi-lgn01.nchc.org.tw:/home/u4439705/run_predict.slurm ~/Desktop/
```

(註：路徑需根據你檔案實際位置微調，若在 /work 下則路徑要改)

方法 B：使用 VS Code（最強推）

如果你桌面有安裝 **VS Code**，建議安裝 **"Remote - SSH"** 擴充功能：

1. 連線到 cbi-lgn01.nchc.org.tw。
2. 在左側檔案瀏覽器找到檔案，點右鍵選 **"Download..."**。

⚠️ 在你貼出內容前，我的初步預測：

既然目前的進度是 13 小時才跑 136 筆，我猜測舊腳本（Gemini 3.1 版）可能犯了兩個錯誤：

- **沒有啟用 Pipeline 的並行能力**：模型在等待前一筆 JSON 寫完才開始下一筆。
- **Token 限制過大且無停止符**：對於 27B 這種模型，如果沒設停止符，它在寫完 JSON 後可能會開始「寫註解」或「解釋代碼」，直到把 max_new_tokens 燒完為止。

請把那兩份檔案內容貼給我，我們來進行一場深度「代碼審查（Code Review）」，把那預計 7 天的工期縮短到 10 小時內！

你想先貼 predict_custom.py 還是 run_predict.slurm？

You

```
(base) [u4439705@cbi-lgn01 u4439705]$ cat predict_custom.py import torch from transformers import AutoModelForCausalLM, AutoTokenizer, TextStreamer, BitsAndBytesConfig from peft import PeftModel import json import functools
```

強制所有 print 立即輸出，不寫入緩衝區

```
print = functools.partial(print, flush=True)
```

===== 設定區 =====

```
BASE_MODEL_PATH = "/work/u4439705/models/Qwen3.5-27B" LORA_PATH = "/work/u4439705/n8n_qwen35_lora" TEST_DATA =
"/work/u4439705/data/n8n_training_data_v2_humanized.jsonl" OUTPUT_FILE = "/work/u4439705/n8n_predicted_workflows.jsonl"

print("載入 Tokenizer...") tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL_PATH, trust_remote_code=True)

print("載入 Base Model (4-bit 量化)...") bnb_config = BitsAndBytesConfig( load_in_4bit=True, bnb_4bit_compute_dtype=torch.bfloat16, )

base_model = AutoModelForCausalLM.from_pretrained( BASE_MODEL_PATH, device_map="auto", trust_remote_code=True, torch_dtype=torch.bfloat16,
quantization_config=bnb_config, )

print("載入 LoRA Adapter ({LORA_PATH})...") model = PeftModel.from_pretrained(base_model, LORA_PATH) model.eval()
```

===== 讀取測試資料 =====

```
with open(TEST_DATA, "r", encoding="utf-8") as f: lines = f.readlines()
```

```
print("\n" + "="*50) print(f'🚀 開始批次測試推理 (共 {len(lines)} 筆)') print("="*50 + "\n")
```

用於將產生的結果即時寫入檔案

```
with open(OUTPUT_FILE, "w", encoding="utf-8") as out_f:    for idx, line in enumerate(lines):        item = json.loads(line)        instruction = item.get("instruction", "")        user_input = item.get("input", "")

user_content = instruction + "\n" + user_input if instruction else user_input

messages = [        {"role": "system", "content": "You are Qwen, created by Alibaba Cloud. You are a helpful assistant."},        {"role": "user", "content": user_content}    ]

# 使用手動組裝或調整 apply_chat_template 來逼迫模型直接輸出 JSON    prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)    # 強制模型以 json 開頭作答，跳過思考過程    prompt += "json\n"

inputs = tokenizer(prompt, return_tensors="pt").to("cuda")

print(f"\n🔄 處理案例 #{idx} ({idx+1}/{len(lines)})")    # 只印前 100 字避免洗版    print(f'【輸入需求】: {user_input[:100]}...')

with torch.no_grad():    output_ids = model.generate(**inputs, max_new_tokens=4096, # 退回我們 100% 確定能跑的安全值    temperature=0.2, top_p=0.9, repetition_penalty=1.05, pad_token_id=tokenizer.eos_token_id, use_cache=True, # 確認使用 KV Cache    )

# 檢查是否因為達到 max_new_tokens 而被截斷    if output_ids.shape[1] - inputs["input_ids"].shape[1] >= 4096:        print("⚠️ 警告：產生的回覆可能被截斷 (達到 4096 限制) !")        is_truncated = True    else:        is_truncated = False

# 只獲取新生成的 token    generated_ids = output_ids[0][len(inputs["input_ids"])[0]:]    output_text = tokenizer.decode(generated_ids, skip_special_tokens=True)

# 存檔    result_item = {        "index": idx,        "input": user_input,        "predicted_output": output_text,        "is_truncated": is_truncated    }    out_f.write(json.dumps(result_item, ensure_ascii=False) + "\n")    out_f.flush() # 每跑完一筆就立刻寫入硬碟，即使中途意外中斷也不會遺失

print(f'✅ 完成並寫入檔案。長度: {len(output_text)} 字元')

# 關鍵：強制清空 VRAM Cache，防止連續生成導致 OOM    del output_ids    del inputs    torch.cuda.empty_cache()

print(f"\n🎉 全部完成！結果已儲存至 {OUTPUT_FILE}") (base) [u4439705@cbi-lgn01 u4439705]$ cat run_predict.slurm #!/bin/bash #SBATCH -A MST114504 #SBATCH -J predict_qwen35 #SBATCH -p normal2 #SBATCH --nodes=1 #SBATCH --gres=gpu:1 #SBATCH --cpus-per-task=8 #SBATCH --mem=80G #SBATCH --output=predict_%j.log cd /work/u4439705 source ~/miniconda3/etc/profile.d/conda.sh conda activate qwen35 python predict_custom.py
```

Gemini

看完這兩份腳本後，身為 CS 同行，我終於抓到為什麼你的 H200 會跑出「家用電腦」速度的元兇了。

這份腳本（predict_custom.py）確實寫得很紮實，但在 **H200** 這種頂級算力環境下，它犯了幾個「性能自殺」的錯誤。

🔍 性能診斷：為什麼每筆要跑 6 分鐘？

1. 致命傷：torch.cuda.empty_cache()

在迴圈的最後一行，你寫了 torch.cuda.empty_cache()。

- **原因：**這是一個非常昂貴的操作。它會強制同步 CPU 與 GPU，並讓 GPU 的記憶體管理器重新規劃。
- **後果：**在迴圈裡放這行，就像是你人在高速公路每開 100 公尺就熄火重新發動一次。這會讓 H200 的頻寬優勢完全消失。

2. 量化開銷：4-bit (BNB) 在 H200 上反而慢

- **分析：**你用了 load_in_4bit=True。這通常是給記憶體不足（例如只有 24GB 的 RTX 3090/4090）用的。
- **後果：**H200 有 **141GB** 的顯存。Qwen 3.5 27B 即使跑全精度（BF16）也才佔約 54GB。在 H200 上跑 4-bit，GPU 必須在運算時不斷進行「反量化（Dequantization）」，這會增加額外的計算負擔，反而比直接跑 BF16 慢。

3. 缺乏 Flash Attention 2

- **分析：**腳本中沒有明確指定 attn_implementation="flash_attention_2"。
- **後果：**處理像 n8n JSON 這種長文本時，標準的 Attention 運算複雜度是平方級。沒有 Flash Attention，處理速度會隨著生成的 JSON 越長而呈指數級變慢。

4. 單筆序列執行 (Batch Size = 1)

- **分析：**這是一筆筆 for 迴圈處理。
- **後果：**GPU 的 Tensor Core 大部分時間都在閒置等數據。

🚀 修正方案：H200 專用 Turbo 版 predict_custom_v2.py

為了讓速度提升 **10 倍以上**，我們要做以下修改：

1. 移除 empty_cache()。
2. 捨棄 4-bit，改用 BF16 原生精度（H200 記憶體絕對夠）。
3. 強制開啟 Flash Attention 2。
4. 引入 Pipeline Batching（雖然 vLLM 不能用，但 transformers 的批次功能還是很強）。

🔧 建議修改後的代碼架構：

```

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline
from peft import PeftModel
import json

# ===== 設定區 =====
BASE_MODEL_PATH = "/work/u4439705/models/Qwen3.5-27B"
LORA_PATH = "/work/u4439705/n8n_qwen35_lora"
TEST_DATA = "/work/u4439705/data/n8n_training_data_v2_humanized.jsonl"
OUTPUT_FILE = "/work/u4439705/n8n_predicted_workflows_fast.jsonl"

print("🚀 正在載入 Tokenizer 與 BF16 模型 (Flash Attention 2)...")
tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL_PATH, trust_remote_code=True)
# 設定 Padding Token 防止 Batch 推理出錯
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "left"

base_model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL_PATH,
    device_map="auto",
    torch_dtype=torch.bfloat16, # H200 直跑 BF16 最快
    trust_remote_code=True,
    attn_implementation="flash_attention_2" # 關鍵：大幅加速長文本
)

print(f"🔗 載入 LoRA Adapter...")
model = PeftModel.from_pretrained(base_model, LORA_PATH)
model = model.merge_and_unload() # 合併權重可以讓推理更快
model.eval()

# 建立高效 Pipeline
pipe = pipeline(
    "text-generation",
    model=model,
    tokenizer=tokenizer,
    device_map="auto"
)

# ===== 批次處理邏輯 =====
with open(TEST_DATA, "r", encoding="utf-8") as f:
    inputs = [json.loads(line)["input"] for line in f]

print(f"🔥 開始並行推理 (Batch Size = 8)...")
# 利用 Pipeline 的批次功能，充分壓榨 H200 的頻寬
# 如果顯存還有剩，batch_size 可以上調到 16
with open(OUTPUT_FILE, "w", encoding="utf-8") as out_f:
    for i, out in enumerate(pipe(inputs, batch_size=8, max_new_tokens=2048, temperature=0.2, top_p=0.9)):
        generated_text = out[0]['generated_text']

        # 簡單清洗並存檔
        result = {"index": i, "predicted_output": generated_text}
        out_f.write(json.dumps(result, ensure_ascii=False) + "\n")

    if i % 10 == 0:
        print(f"✅ 已完成 {i}/{len(inputs)} 筆")

print("🎉 任務成功完成！")

```

Slurm 調整建議 (run_predict.slurm)

你的 Slurm 設定基本正確，但因為我們現在要跑「全馬力 BF16」，建議調整記憶體：

```

#SBATCH --mem=120G          # H200 節點記憶體很足，給多一點防止載入模型時抖動
#SBATCH --cpus-per-task=16 # 增加 CPU 數量來加速 Tokenizer 的處理

```

💡 最後的建議

1. **果斷停止舊任務**：scancel 134267，它真的太慢了。
2. **合併 LoRA**：我在腳本中加了 merge_and_unload()，這會把 LoRA 權重融合進 Base Model，能進一步提升推理速度。
3. **檢查 Flash Attention**：確保你的環境有安裝 flash-attn 套件。如果沒有，pip install flash-attn --no-build-isolation。

如果你把這份「Turbo 版」跑起來，這 1,963 筆數據應該能在 2 - 4 小時內收工，而不是 8 天。你要現在幫你微調這份腳本中的 Prompt 格式，以確保它能完美銜接你原本的 XML 結構嗎？